

Local Job Finder

Cross-Platform Mobile App Development Project

Flutter · Provider · Hive · REST API — Project Brief

1. Project Objectives

Design and build **Local Job Finder**, a complete cross-platform mobile application for the Indian job market, using Flutter. The app lets job seekers browse, search, filter, save, and apply to listings, and lets employers post and manage their own openings. The project is meant to take a team from a blank Flutter project to a working, well-architected app backed by a real REST API — covering UI/UX design, state management, local persistence, networking, and error handling along the way.

By the end of the project, each team should be able to demonstrate:

- A responsive Flutter UI that adapts across phone, tablet, and desktop breakpoints.
- Clean state management using the Provider pattern, separated from UI code.
- Full CRUD operations (create, read, update, delete) on job listings.
- Integration with a real, running REST API — not a hardcoded dataset.
- Local persistence (Hive + SharedPreferences) for offline-friendly behavior.
- Thoughtful error handling and loading states, not just the happy path.

2. Team Formation

Teams of **3–4 students**. Each team self-organizes and assigns an owner for each area below; owners are accountable for their area's quality but everyone is expected to read and understand the whole codebase.

Role	Primary Ownership	Typical Files
UI / Screens Lead	Screen layout, navigation, responsive design	lib/screens/*, utils/app_theme.dart
State & Data Lead	Provider state, models, filtering logic	lib/services/job_provider.dart, models/*
Backend & Persistence Lead	REST API integration, Hive caching, auth	lib/services/api_service.dart, hive_service.dart
QA / Docs Lead	Testing, validation, README, submission package	test/*, README.md

Teams of 3 may merge the QA/Docs role into another; roles may also rotate at the project's midpoint so every member touches both frontend and backend code.

3. Weekly Timeline

Week	Focus	Milestone
1	Setup & planning	Repo created, Flutter environment running, team roles assigned, wireframes for all 10 screens
2	Core UI	Splash, Onboarding, Login, Sign Up, and Home screens built with static/mock data
3	State management	Provider wired up; search, category/city/type filters, and responsive grid working
4	Persistence & CRUD	Hive integrated; Save, Apply, and Post/Edit/Delete Job flows fully working locally
5	API integration	App connected to a real REST backend (see server/); loading, error, and empty states implemented
6	Polish & submission	Bug fixes, responsive QA pass, README finalized, demo video recorded, final submission packaged

Each week ends with a short team check-in (async post or 10-minute call) confirming the milestone was hit before moving on.

4. Deliverables

Each team submits the following:

- **Source code repository** — full Flutter project (lib/, assets/, pubspec.yaml) plus the server/ directory containing the REST API the app talks to.
- **Ten working screens**: Splash, Onboarding, Login, Sign Up, Home, Job Details, Post/Edit Job, Saved Jobs, Applied Jobs, and Profile.
- **README.md** documenting setup steps, architecture decisions, and how to run both the app and the API server.
- **Short demo video (3–5 minutes)** walking through the core flows: browsing/filtering, saving a job, applying, and posting/editing/deleting a job as an employer.
- **Individual reflection** (one paragraph per member) on what they owned and what they'd do differently.

5. Evaluation Criteria

Criterion	Weight	What's assessed
Functionality	30%	All 10 screens work; CRUD, save, and apply flows behave correctly end-to-end
Architecture & Code Quality	20%	Clear separation of UI / state / services; readable, idiomatic Dart
API & Data Integration	20%	Real REST calls (not hardcoded data), sensible error/loading/empty states
UI/UX & Responsiveness	15%	Consistent design system, adapts across screen sizes, good empty/error states
Persistence & Offline Behavior	10%	Hive caching works; app remains usable without a live connection
Documentation & Presentation	5%	README clarity, demo video quality, commit history

6. Submission Checklist

- [] Repository is public (or shared with instructors) and builds from a clean clone
- [] `flutter pub get` and `flutter run` work without manual fixes
- [] API server starts with `npm install` & `npm start` and the app connects to it by default
- [] App still functions if the API server is not running (offline fallback verified)
- [] All 10 required screens are present and reachable via normal navigation
- [] Form validation works on Login, Sign Up, and Post/Edit Job screens
- [] README includes setup instructions for both the Flutter app and the server
- [] Demo video uploaded and linked in the README
- [] Individual reflections included

[] No API keys, secrets, or personal data committed to the repository

7. General Guidelines

Working conventions

- Use feature branches and pull requests, even within the team — it keeps history reviewable.
- Keep UI (screens/, widgets/), state (services/), and data (models/) cleanly separated.
- Follow the existing formatting/lint rules (`flutter_lints`) — run `flutter analyze` before pushing.
- Commit early and often; a thin commit history the night before the deadline is a red flag.

Working with the API

- Treat `assets/jobs.json` as seed/offline data only — the app's source of truth is the live API.
- Handle network failures gracefully: show a retry action, never a blank screen or crash.
- Don't block the UI thread on network calls; use `async/await` and loading indicators consistently.

Academic integrity

- AI tools may be used for boilerplate and debugging help, but every member must be able to explain any code in the submission.
- Cite any third-party packages or code snippets beyond what's listed in `pubspec.yaml`.

8. Expected Learning Outcomes

By completing this project, students will be able to:

- Build a multi-screen Flutter application with proper navigation and responsive layouts.
- Apply the Provider pattern to manage shared app state cleanly, outside the widget tree.
- Design and consume a REST API, including handling latency, errors, and empty states.
- Persist data locally with Hive and SharedPreferences, and reconcile local vs. remote state.
- Implement full CRUD functionality across a real data model.
- Write and validate forms with meaningful, user-facing error messages.
- Collaborate on a shared codebase using version control and role-based ownership.
- Communicate technical work clearly through documentation and a live demo.